

Intermediate (test a tool without calling the LLM)

— Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper pytest), **Advanced** (what changes once the tool touches a real network call). Read Basic first even if you already know pytest — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A "tool" from `06_tools_function_calling` is just a normal Python function underneath — the model only decides *when* to call it. The function's own logic (what it does once called) doesn't need a model at all to test.

So: stop thinking about the LLM completely for this exercise. Pick one tool function, call it directly yourself with made-up arguments, and check what it returns — the same way you'd test any other function.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

This exercise is deliberately separating two questions that get blurred together if you only ever test through the full agent:

1. Does the tool function itself do the right thing, given specific arguments?
2. Does the model choose to call it, with the right arguments, at the right time?

Question 1 has nothing to do with the LLM and can be tested with a normal, fast, exact pytest test — no API key, no network call, no cost, no flakiness. Question 2 is a separate, harder problem (closer to the Real-world exercise later in this document) that you should test separately, not mixed into the same test.

The exact pieces:

- **Import the function directly** — `from tools import get_weather` — treating it exactly like any other importable function, because that's all it is.
- **Call it with hardcoded arguments** that represent a real, expected case: `result = get_weather(city="Lahore")`.
- **Assert on the structure and values you actually control** — if the tool returns a dict, check the keys you expect exist and hold sensible values, not just that *something* came back.
- **No API key needed** — this test never touches the model or any real credentials.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Some tools don't just compute something in memory — they call a real external API (a real weather service, a real database, a real payment gateway). Calling the *real* thing from a test is slow, costs money or rate-limit budget, can fail for reasons that have nothing to do with your code (the third-party service is just down), and makes your test non-deterministic — exactly the "flaky test" problem this document's Edge cases exercise deals with, but from a different cause.

The fix isn't to skip testing that tool. It's to **fake the network call, but keep testing your own code around it** — the part that builds the request, parses the response, and handles a bad status code. That's `unittest.mock.patch` : it temporarily replaces one function or method with a fake one for the duration of a test, then puts the real one back automatically afterward.

The real design question: what's actually *yours* to test here? Not "does the weather API work" (it's not your code, and it has its own tests) — but "does my function build the right request, and does it handle a non-200 response the way I intended, instead of crashing or silently returning garbage?"

The extra pieces needed:

- `from unittest.mock import patch — @patch("tools.requests.get")` (patch the name *where it's used*, i.e. inside `tools.py`, not where `requests` itself is defined) replaces the real network call with a fake one just for that test.
- `mock_get.return_value = <a fake response object>` — controls exactly what the faked call returns, so you can test both the success path and a simulated failure (a 500 status, a timeout) without any real network access.
- A separate, clearly-marked **integration test** (maybe skipped by default, e.g. `@pytest.mark.skip(reason="hits real API, run manually")`) for the rare case you actually do want to hit the real service once in a while, kept apart from the fast unit-style tests that run on every commit.

Sketch what you'd fake `requests.get` to return for both a success and a failure case, before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both assume the tool is pure logic — call it, check the return value, done. Advanced covers the case that assumption breaks: a tool that calls out to a real network resource. The fix isn't a different kind of assert, it's faking the one part of the tool that isn't actually yours (the third-party API) so you can still test everything that *is* yours — request building, response parsing, error handling — quickly, for free, and every single time.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
import the tool function directly (not through the model)

def test_tool_returns_expected_result():
    result = call the tool with a made-up, hardcoded argument
    assert the result has the shape/value you expect
```

Here's almost the whole thing — just try running it and reading it line by line:

```
from tools import get_weather

def test_get_weather_returns_dict():
    result = get_weather("Lahore")
    assert isinstance(result, dict)
    assert "temperature" in result
```

Adjust the function name and expected keys to your own real tool, then check it runs with `pytest -v`.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
from tools import <your_tool_function>

def test_<tool_name>_returns_expected_result():
    result = <your_tool_function>(<hardcoded arguments>)
    assert isinstance(result, <expected type>)
    assert result[<some key>] == <expected value>

def test_<tool_name>_handles_bad_input():
    result_or_error = call the tool with an obviously bad argument
    assert it either raises the right error, or returns a clear "invalid" result
```

```
from tools import get_weather

def test_get_weather_returns_expected_shape() -> None:
    result = get_weather(city="Lahore")
    assert isinstance(result, dict)
    assert "temperature" in result
    assert isinstance(result["temperature"], (int, float))
```

Notice the second planned test — a tool test isn't just "does it work with good input," it's also "does it fail *usefully* with bad input," since a real model will sometimes call your tool with arguments that don't make sense. Write that bad-input test yourself before moving to Advanced.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
import patch from unittest.mock

def test_get_weather_success(mocked requests.get):
    make the fake response return status_code 200 and a fake weather json body
    call get_weather("Lahore")
    assert the result was built correctly from that fake json

def test_get_weather_api_failure(mocked requests.get):
    make the fake response return status_code 500
    call get_weather("Lahore")
    assert it returns a clear error result, doesn't crash, doesn't return fake su
```

Here's almost the whole thing — fill in the failure case yourself:

```
from unittest.mock import patch, Mock
from tools import get_weather

@patch("tools.requests.get")
def test_get_weather_success(mock_get: Mock) -> None:
    mock_get.return_value.status_code = 200
    mock_get.return_value.json.return_value = {"temp_c": 22, "condition": "Clear"}

    result = get_weather(city="Lahore")

    assert result["temperature"] == 22
    assert "error" not in result

@patch("tools.requests.get")
def test_get_weather_api_failure(mock_get: Mock) -> None:
    mock_get.return_value.status_code = 500
    # your turn: what should get_weather return when the real API fails?
    # it should NOT crash, and it should NOT return a fake-looking success dict
    ...
```

Fill in the failure test yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both call the tool directly and trust that whatever it returns came from real, deterministic logic — true for a pure function, but not true the moment the tool calls a real API. Advanced fakes that one external dependency with `@patch`, so the test still runs in milliseconds with no network access, while still genuinely exercising your request-building and response-parsing code on both a success and a failure response.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)